

2주차 코딩실습

60분 실습 구성

시간	단계	내용
0~10분	데이터 준비	서로 다른 병원 데이터 소스를 하나의 SQLite 테이블로 통합합니다.
10~30분	예제 따라하기	조회, 조건, 정렬, 결측 진단, 그룹 집계, 조건부 집계를 실행합니다.
30~45분	기본 문제	예제 SQL을 변형하여 직접 작성합니다.
45~55분	응용·도전 문제	CASE, HAVING , 서브쿼리로 현실적인 질문을 해결합니다.
55~60분	결과 확인	예상 결과와 비교하고 소스 파일과 실행 화면을 제출합니다.

새 파일 **week02_practice.py**를 만들고 아래 코드를 순서대로 실행합니다. 코드 블록을 한꺼번에 복사하기보다 단계별로 실행하고 VS Code 터미널의 결과를 확인합니다.

데이터 준비

이번 실습에서는 두 병원의 예약 데이터를 통합합니다. 가나병원은 행과 열이 고정된 CSV 형태의 튜플을, 다라병원은 필드가 유연한 JSON 형태의 딕셔너리를 제공합니다. 두 소스는 구조가 다르지만 최종적으로 동일한 **appointments** 테이블에 저장합니다.

1. 연결과 테이블 생성

```
import sqlite3
import random
from datetime import date, timedelta

random.seed(2026)

conn = sqlite3.connect("clinic_sources.db")
conn.row_factory = sqlite3.Row
cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS appointments")
cur.execute("""
CREATE TABLE appointments (
  appt_id    INTEGER PRIMARY KEY,
  hospital   TEXT NOT NULL,
  visit_date TEXT NOT NULL,
  patient_age INTEGER,
  department TEXT NOT NULL,
  wait_min   INTEGER,
  status     TEXT NOT NULL,
  source_type TEXT NOT NULL
)
""")
```

row_factory를 지정하면 조회 결과를 열 이름으로 읽을 수 있습니다. **patient_age**와 **wait_min**은 원천 데이터에서 누락될 수 있으므로 **NOT NULL**을 적용하지 않았습니다.

2. 정형 소스 적재

가나병원 자료는 열 순서가 고정된 CSV 행을 튜플로 읽었다고 가정합니다.

```
departments = ["내과", "정형외과", "소아과", "피부과"]
statuses = ["완료", "완료", "완료", "취소"]
start = date(2026, 8, 1)

gana_rows = []
for appt_id in range(1, 37):
    visit_date = (start + timedelta(days=(appt_id - 1) % 14)).isoformat()
    age = random.randint(5, 82)
    department = random.choice(departments)
    wait_min = max(3, int(random.gauss(24, 10)))
    status = random.choice(statuses)
    gana_rows.append(
        (appt_id, "가나병원", visit_date, age,
         department, wait_min, status, "csv")
    )

cur.executemany("""
INSERT INTO appointments
VALUES (?, ?, ?, ?, ?, ?, ?)
""", gana_rows)
```

executemany()는 동일한 SQL에 여러 행을 한 번에 전달합니다. 값은 문자열 결합이 아니라 ? 플레이스홀더로 전달합니다.

3. 반정형 소스 변환.적재

다라병원 JSON은 필드명이 다르고 일부 레코드에는 **age** 또는 **wait**가 없습니다. **dict.get()**은 키가 없을 때 **None**을 반환하며 SQLite에는 **NULL**로 저장됩니다.

```
dara_json = []
for appt_id in range(37, 73):
    record = {
        "id": appt_id,
        "date": (start + timedelta(days=(appt_id - 37) % 14)).isoformat(),
        "dept": random.choice(departments),
        "state": random.choice(statuses),
        "age": random.randint(5, 82),
        "wait": max(3, int(random.gauss(31, 13)))
    }
    dara_json.append(record)

# 실제 수집에서 흔히 생기는 결측과 이상값을 의도적으로 포함합니다.
for record in dara_json:
    if record["id"] in {41, 58, 69}:
        record.pop("wait")
    if record["id"] in {47, 64}:
        record.pop("age")
dara_json[11]["wait"] = 180
dara_json[25]["age"] = 220

for record in dara_json:
    cur.execute("""
INSERT INTO appointments
VALUES (?, ?, ?, ?, ?, ?, ?)
""", (
        record["id"], "다라병원", record["date"],
        record.get("age"), record.get("dept"), record.get("wait"),
```

```

        record["state"], "json"
    ))

    conn.commit()
    print("appointments 테이블 생성 완료")

```

DROP TABLE IF EXISTS부터 다시 실행하면 항상 같은 72건으로 초기화됩니다. **random.seed(2026)**을 사용했기 때문에 모든 학생에게 동일한 결과가 나옵니다.

예제 코드

예제 1. 테이블 구조와 표본 행 확인

데이터를 받자마자 집계부터 하지 않고 스키마와 표본 행을 먼저 확인합니다.

```

print("== 테이블 구조 ==")
for row in cur.execute("PRAGMA table_info(appointments)"):
    print(row[1], row[2], "NOT NULL=" + str(row[3]))

print("\n== 앞의 5행 ==")
for row in cur.execute("""
SELECT appt_id, hospital, visit_date, department, wait_min, source_type
FROM appointments
ORDER BY appt_id
LIMIT 5
"""):
    print(dict(row))

```

핵심 SQL은 **PRAGMA table_info**, **ORDER BY**, **LIMIT**입니다.

예제 2. 전체 규모와 소스별 건수

```

total = cur.execute("SELECT COUNT(*) FROM appointments").fetchone()[0]
print("전체 예약:", total)

for row in cur.execute("""
SELECT source_type, COUNT(*) AS cnt
FROM appointments
GROUP BY source_type
ORDER BY source_type
"""):
    print(row["source_type"], row["cnt"])

```

예상 결과는 전체 72건, CSV 36건, JSON 36건입니다. **COUNT(*)**은 모든 행을 세고 **GROUP BY**는 같은 소스 유형끼리 묶습니다.

예제 3. 플레이스홀더를 사용한 조건 조회

```

department = "내과"
min_wait = 30

for row in cur.execute("""
SELECT appt_id, hospital, patient_age, wait_min
FROM appointments
WHERE department = ? AND wait_min >= ?
ORDER BY wait_min DESC
""", (department, min_wait)):
    print(dict(row))

```

값을 SQL 문자열에 직접 붙이지 않고 플레이스홀더로 전달하면 따옴표 오류와 SQL 주입 위험을 줄일 수 있습니다.

예제 4. 결측값 진단

COUNT(열)은 **NULL**을 제외하지만 **COUNT(*)**은 모든 행을 셉니다. 두 값의 차이로 결측 개수를 계산합니다.

```
row = cur.execute("""
SELECT
    COUNT(*) AS total,
    COUNT(*) - COUNT(patient_age) AS missing_age,
    COUNT(*) - COUNT(wait_min) AS missing_wait
FROM appointments
""").fetchone()

print(dict(row))
```

예상 결과는 **total=72, missing_age=2, missing_wait=3**입니다.

예제 5. 병원별 요약 통계

```
for row in cur.execute("""
SELECT
    hospital,
    COUNT(*) AS appointments,
    ROUND(AVG(wait_min), 1) AS avg_wait,
    MIN(wait_min) AS min_wait,
    MAX(wait_min) AS max_wait
FROM appointments
GROUP BY hospital
ORDER BY avg_wait DESC
"""):
    print(dict(row))
```

AVG, MIN, MAX는 **NULL**을 자동으로 제외합니다. 평균을 해석할 때는 결측 건수도 함께 확인해야 합니다.

예제 6. 조건부 집계

```
for row in cur.execute("""
SELECT
    hospital,
    COUNT(*) AS total,
    SUM(CASE WHEN status = '취소' THEN 1 ELSE 0 END) AS canceled,
    SUM(CASE WHEN wait_min IS NULL THEN 1 ELSE 0 END) AS missing_wait,
    SUM(CASE WHEN wait_min >= 60 THEN 1 ELSE 0 END) AS wait_60_plus
FROM appointments
GROUP BY hospital
ORDER BY hospital
"""):
    print(dict(row))
```

CASE WHEN을 집계 함수 안에 넣으면 여러 품질 조건을 한 SQL로 셀 수 있습니다.

예제 7. 유효성 위반 찾기

나이는 0~120세, 대기시간은 0~120분이 정상이라고 가정합니다.

```
for row in cur.execute("""
SELECT appt_id, hospital, patient_age, wait_min
FROM appointments
WHERE (patient_age IS NOT NULL AND patient_age NOT BETWEEN 0 AND 120)
    OR (wait_min IS NOT NULL AND wait_min NOT BETWEEN 0 AND 120)
"""):
    print(dict(row))
```

```
ORDER BY appt_id
""");
print(dict(row))
```

대기시간 180분과 나이 220세인 두 행이 탐지됩니다. 발견 즉시 삭제하지 않고 원천 시스템과 업무 규칙을 먼저 확인합니다.

코드 문제

위의 데이터 준비 코드를 실행한 상태에서 아래 문제를 풉니다. 각 SQL을 **week02_practice.py**에 추가하고 예상 결과와 비교한 뒤 정답·해설을 펼칩니다.

기본 — 조회와 집계

문제 1. 진료과별 예약 건수

department별 예약 건수를 구하고 건수가 많은 순서로 출력합니다.

- 사용할 열: **department, COUNT(*) AS cnt**
- 사용할 구문: **GROUP BY, ORDER BY**
- 예상 결과: 진료과 4개가 출력되고 전체 **cnt**의 합은 72입니다.

TODO: 진료과별 예약 건수를 출력하세요.

문제 1 정답·해설

```
for row in cur.execute("""
SELECT department, COUNT(*) AS cnt
FROM appointments
GROUP BY department
ORDER BY cnt DESC, department
"""):
    print(dict(row))
```

GROUP BY department가 같은 진료과를 묶고 **COUNT(*)**이 각 그룹의 행 수를 셉니다. 동렬일 때도 결과가 일정하도록 두 번째 정렬 기준을 넣었습니다.

문제 2. 특정 기간 예약 조회

2026년 8월 5일부터 8월 10일까지의 완료된 예약을 날짜·예약번호 순서로 출력합니다.

- 출력 열: **appt_id, visit_date, hospital, department, status**
- 사용할 구문: **BETWEEN, AND, ORDER BY**
- 예상 결과: 모든 행의 날짜가 지정 범위 안이고 상태가 **완료**입니다.

TODO: 날짜 범위와 status 조건을 모두 적용하세요.

문제 2 정답·해설

```

for row in cur.execute("""
SELECT appt_id, visit_date, hospital, department, status
FROM appointments
WHERE visit_date BETWEEN ? AND ?
      AND status = ?
ORDER BY visit_date, appt_id
""", ("2026-08-05", "2026-08-10", "완료")):
    print(dict(row))

```

날짜가 **YYYY-MM-DD** 형식으로 통일되어 있으므로 문자열 비교가 가능합니다. **BETWEEN**은 시작과 끝도 포함합니다.

문제 3. 병원별 결측률

병원별 전체 건수, 나이 결측 건수, 대기시간 결측 건수, 대기시간 결측률을 출력합니다.

- 결측률: $\text{결측 건수} \times 100.0 / \text{전체 건수}$
- **ROUND(..., 1)**로 소수 첫째 자리까지 표시합니다.
- 예상 결과: 가나병원의 결측은 0건이고 다라병원에는 나이 2건·대기시간 3건이 있습니다.

TODO: COUNT와 SUM(CASE WHEN ...)으로 결측 리포트를 작성하세요.

문제 3 정답·해설

```

for row in cur.execute("""
SELECT
    hospital,
    COUNT(*) AS total,
    SUM(CASE WHEN patient_age IS NULL THEN 1 ELSE 0 END) AS missing_age,
    SUM(CASE WHEN wait_min IS NULL THEN 1 ELSE 0 END) AS missing_wait,
    ROUND(SUM(CASE WHEN wait_min IS NULL THEN 1 ELSE 0 END)
          * 100.0 / COUNT(*), 1) AS missing_wait_pct
FROM appointments
GROUP BY hospital
ORDER BY hospital
"""):
    print(dict(row))

```

100.0을 곱하면 실수 비율을 얻습니다. 결측률은 소스 품질을 비교하는 기본 지표입니다.

응용 — 조건과 집계를 조합

문제 4. 평균 대기시간이 긴 진료과

진료과별 평균 대기시간과 예약 건수를 구하고 평균 대기시간이 25분 이상인 진료과만 출력합니다.

- 사용할 구문: **GROUP BY, HAVING, AVG, ROUND**
- **WHERE**가 아니라 **HAVING**을 사용합니다.

TODO: 그룹 집계 결과를 HAVING으로 필터링하세요.

문제 4 정답·해설

```

for row in cur.execute("""
SELECT department, COUNT(*) AS cnt,
       ROUND(AVG(wait_min), 1) AS avg_wait
FROM appointments
GROUP BY department
HAVING AVG(wait_min) >= 25
ORDER BY avg_wait DESC
"""):
    print(dict(row))

```

WHERE는 그룹화 전 개별 행을, **HAVING**은 그룹화 후 집계 결과를 필터링합니다.

문제 5. 연령대별 예약 분포

환자 나이를 **아동·청소년**(20세 미만), **청년**(20~39세), **중장년**(40~64세), **고령**(65세 이상), **미상**으로 나누고 건수와 평균 대기시간을 출력합니다.

TODO: CASE WHEN으로 age_group을 만든 뒤 GROUP BY 하세요.

문제 5 정답·해설

```

for row in cur.execute("""
SELECT
    CASE
        WHEN patient_age IS NULL THEN '미상'
        WHEN patient_age < 20 THEN '아동·청소년'
        WHEN patient_age < 40 THEN '청년'
        WHEN patient_age < 65 THEN '중장년'
        ELSE '고령'
    END AS age_group,
    COUNT(*) AS cnt,
    ROUND(AVG(wait_min), 1) AS avg_wait
FROM appointments
GROUP BY age_group
ORDER BY age_group
"""):
    print(dict(row))

```

CASE는 원본을 바꾸지 않고 분석용 범주를 만듭니다. **NULL**을 먼저 처리해야 의도하지 않은 분류를 막을 수 있습니다.

문제 6. 전체 평균보다 오래 기다린 예약

전체 평균보다 오래 기다린 예약을 대기시간이 긴 순서로 최대 10건 출력합니다.

TODO: WHERE 절 오른쪽에 전체 AVG를 구하는 서브쿼리를 작성하세요.

문제 6 정답·해설

```

for row in cur.execute("""
SELECT appt_id, hospital, department, wait_min
FROM appointments
WHERE wait_min > (SELECT AVG(wait_min) FROM appointments)
ORDER BY wait_min DESC
LIMIT 10
"""):
    print(dict(row))

```

서브쿼리는 전체 평균이라는 값 하나를 반환합니다. **NULL**은 비교 결과가 참이 아니므로 자동으로 제외됩니다.

도전 — 품질 리포트 완성

문제 7. 소스 유형별 품질 리포트

CSV와 JSON 소스를 비교하는 품질 리포트를 하나의 SELECT 문으로 작성합니다. **source_type**, 전체 건수, 나이·대기시간 결측 건수, 나이·대기시간 범위 위반 건수, 정상 행 건수를 출력합니다. 정상 행은 두 값이 모두 존재하고 0~120 범위 안인 행입니다.

TODO: SUM(CASE WHEN ...)을 여러 번 사용해 품질 리포트를 완성하세요.

문제 7 정답·해설

```
for row in cur.execute("""
SELECT
    source_type,
    COUNT(*) AS total,
    SUM(CASE WHEN patient_age IS NULL THEN 1 ELSE 0 END) AS missing_age,
    SUM(CASE WHEN wait_min IS NULL THEN 1 ELSE 0 END) AS missing_wait,
    SUM(CASE WHEN patient_age IS NOT NULL
        AND patient_age NOT BETWEEN 0 AND 120 THEN 1 ELSE 0 END) AS invalid_age,
    SUM(CASE WHEN wait_min IS NOT NULL
        AND wait_min NOT BETWEEN 0 AND 120 THEN 1 ELSE 0 END) AS invalid_wait,
    SUM(CASE WHEN patient_age BETWEEN 0 AND 120
        AND wait_min BETWEEN 0 AND 120 THEN 1 ELSE 0 END) AS valid_rows
FROM appointments
GROUP BY source_type
ORDER BY source_type
"""):
    print(dict(row))
```

여러 품질 조건을 **SUM(CASE WHEN ...)**으로 표현하면 한 번의 조회로 완전성·유효성 지표를 함께 얻을 수 있습니다.

제출

1. 완성한 **week02_practice.py**
2. 문제 3의 병원별 결측 리포트 실행 화면
3. 문제 7의 소스 유형별 품질 리포트 실행 화면

```
conn.close()
```